Modern Cryptography

Computational Number Theory

Computational Number Theory

- Intro
- Diffie-Hellman key exchange
- Groups
- Extended GCD
- Square-and-Multiply

Intro

Symmetric Key

Symmetric Key (대칭 키)

방식: 암호화 키 = 복호화 키

장점: 암복호화 속도가 빠름

단점: 통신 상대마다 키가 필요해 관리가 어렵고(n(n-1)/2 개), 키 전달 과정에서 탈취 위험이 있다

용도: 대량의 데이터 암호화(파일 저장 등)

SYMMETRIC KEY EXCHANGE: PROVING NECESSARY KEY CALCULATION

Proof: Combination($_nC_2$)

$$_nC_2 = \binom{n}{2}$$

$$_nC_2 = \frac{n!}{2!(n-2)!}$$

$$_nC_2 = \frac{n \times (n-1) \times \cancel{(n-2)} \times \cancel{(n-3)} \times \cdots \times 1}{(2 \times 1) \times (\cancel{(n-2)} \times \cancel{(n-3)} \times \cdots \times 1)}$$

$$_nC_2 = \frac{n \times (n-1)}{2 \times 1}$$

$$\boxed{\text{TOTAL NECESSARY KEYS} = \frac{n(n-1)}{2}}$$

Public Key (공개 키)

방식: 암호화 키(공개) ≠ 복호화 키(비밀)

장점: 키 분배가 필요 없고, 공개키는 노출되어도 무방하여 안전함

단점: 대칭 키에 비해 속도가 매우 느림(수백~수만 배)

용도: 키 교환, 전자 서명

실제로는?

속도가 빠른 대칭 키로 실제 데이터를 암호화하고, 그 대칭 키를 안전하게 공유하기 위해 공개 키 방식을 사용하는 **하이브리드 방식(Ex: HTTPS/SSL)**을 주로 사용한다.

Diffie-Hellman key exchange

Diffie-Hellman key exchange

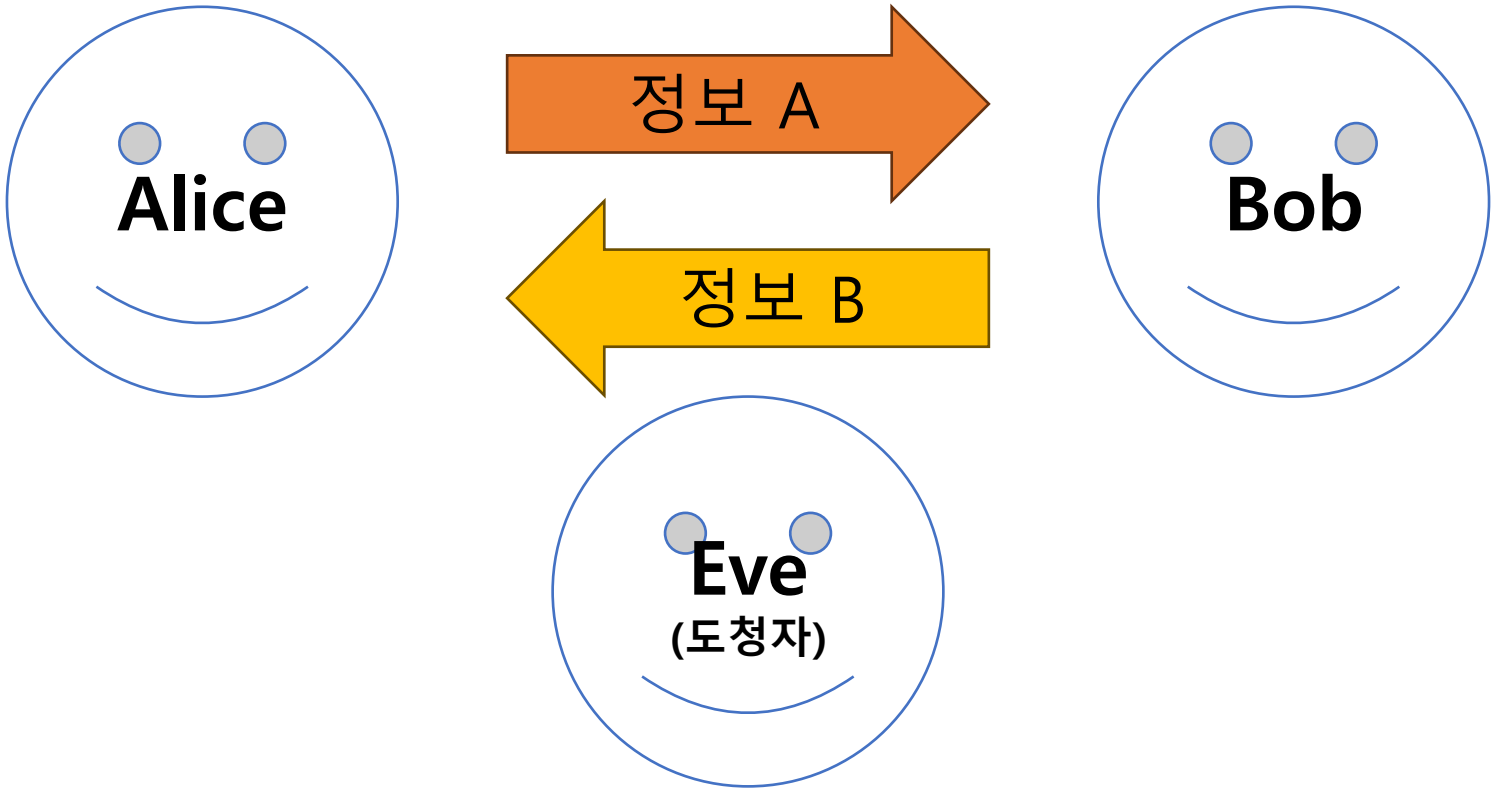
비밀 키 생성: 데이터를 직접 암호화하는 것이 아니라, 대칭 키 암호화에 사용할 비밀 키를 안전하게 만드는 것.

이산대수 문제: 거대한 숫자의 거듭제곱 계산은 쉽지만, 결과값에서 지수(개인 키)를 찾아내는 것은 매우 어렵다는 수학적 원리에 기반.

중간자 공격 취약: 상대방의 신원을 확인하는 인증 절차가 없기 때문에 중간자 공격에 노출될 수 있어 현대에는 디지털 서명 등과 함께 사용됨.

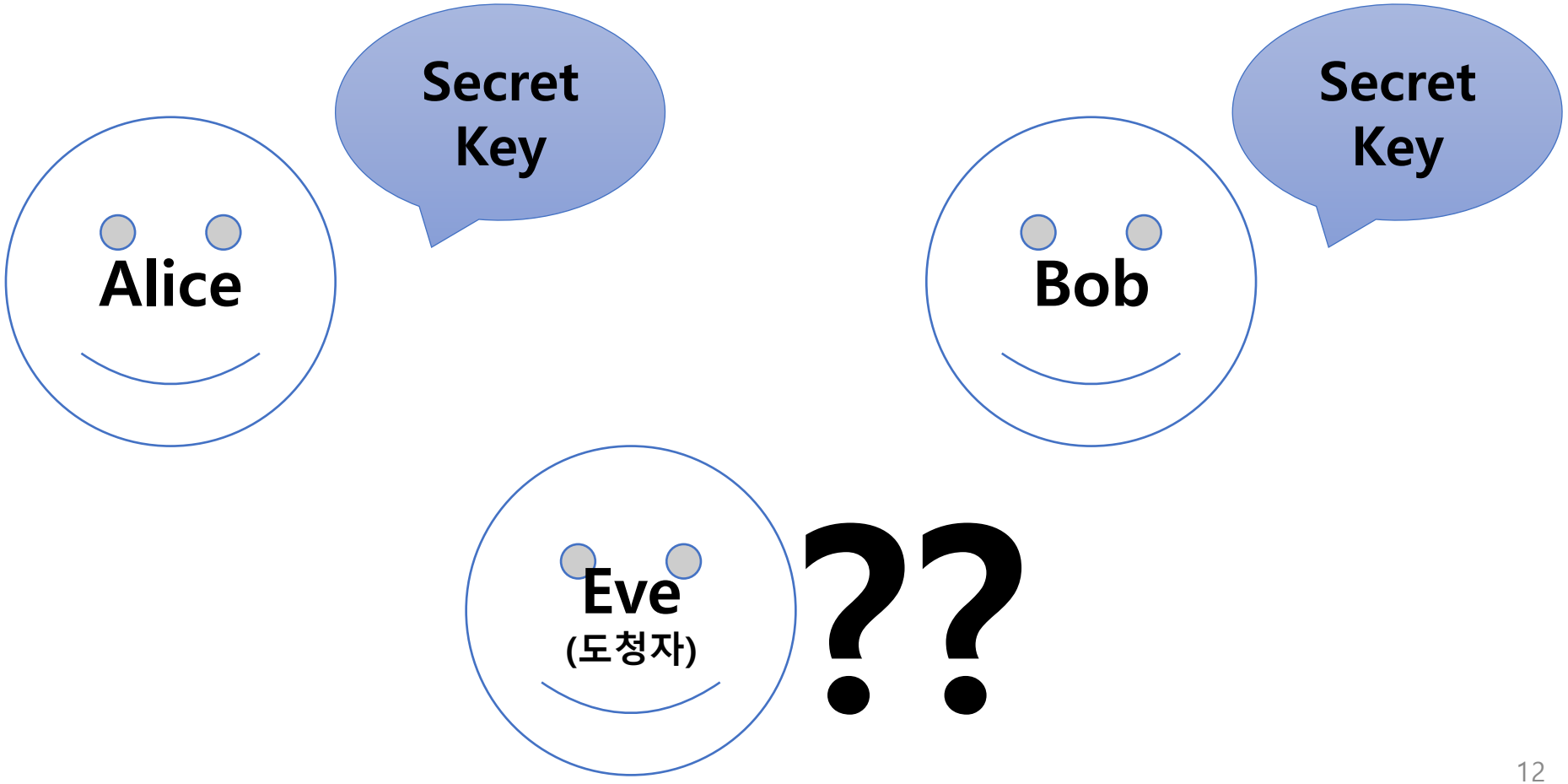
Diffie-Hellman Key Exchange

: Alice 와 Bob이 하는 말을 모두가 들을 수 있는 상황에서 둘만 아는 Secret Key를 만드는 방법



Diffie-Hellman Key Exchange

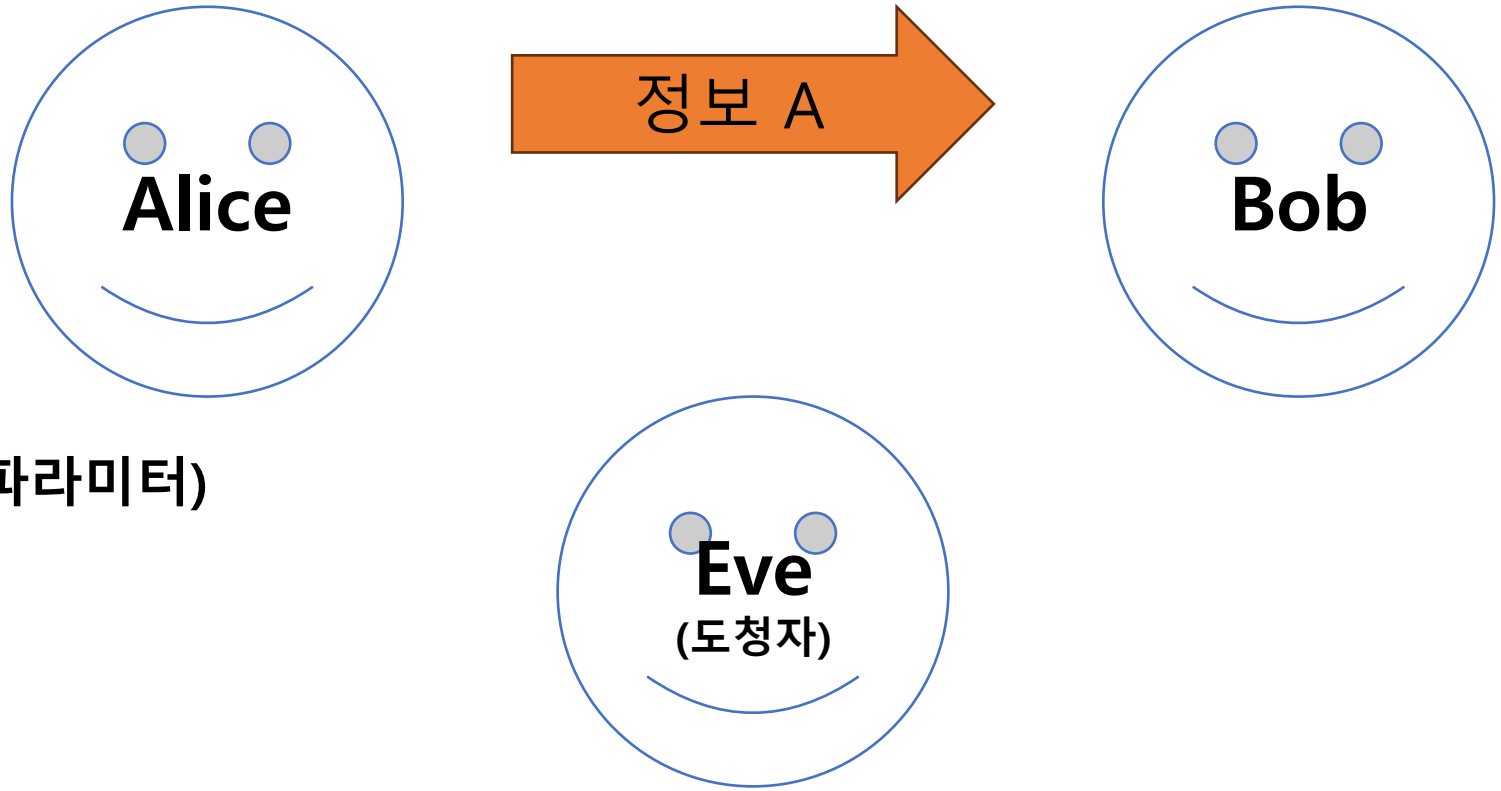
: Alice 와 Bob이 하는 말을 모두가 들을 수 있는 상황에서 둘만 아는 Secret Key를 만드는 방법



Diffie-Hellman Key Exchange

P = 23
g = 5
g^a mod p = 8

P = 23
g = 5
a = 6



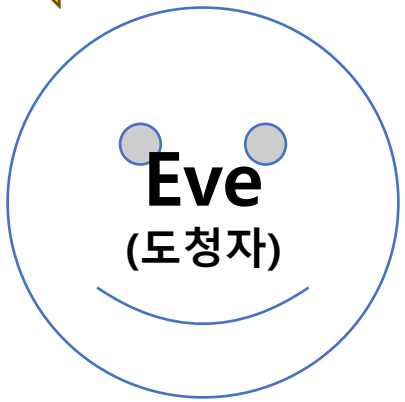
(p, g는 공개 파라미터)

Diffie-Hellman Key Exchange

$g^b \bmod p = 10$

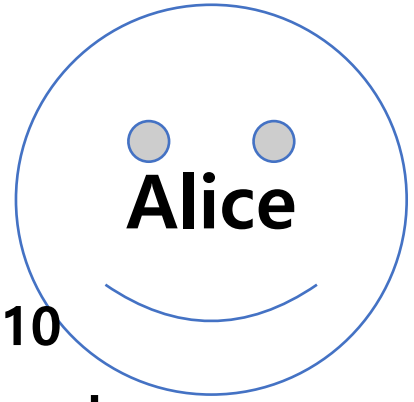
P = 23
g = 5
a = 6

P = 23
g = 5
$g^a \bmod p = 8$
b = 3



P = 23
g = 5
$g^a \bmod p = 8$

Diffie-Hellman Key Exchange



$P = 23$
$g = 5$
$a = 6$
$g^b \bmod p = 10$

$(g^b \bmod p)^a \bmod p$
$= (10)^6 \bmod 23$
$= 6$(secret key)
$= g^{ba} \bmod p$

$P = 23$
$g = 5$
$g^a \bmod p = 8$
$g^b \bmod p = 10$

$P = 23$
$g = 5$
$g^a \bmod p = 8$
$b = 3$

$(g^a \bmod p)^b \bmod p$
$= (8)^3 \bmod 23$
$= 6$(secret key)
$= g^{ab} \bmod p$

Diffie-Hellman Key Exchange

Eve는 a와 b를 모르기 때문에
같은 계산을 하려면
5^x mod 23 = 8 혹은 5^x mod 23 = 10을 만족하는 x를 찾아야 함.
거대한 숫자의 거듭제곱 계산은 쉽지만, 결과값에서 지수(개인 키)를 찾아내는 것은 매우 어렵다는 수학적 원리(이산대수 문제)에 기반.

Alice

$g = 5$
$a = 6$
$g^b \bmod p = 10$

$(g^b \bmod p)^a \bmod p$
$= (10)^6 \bmod 23$
$= 6$
$= g^{ba} \bmod p$

Bob

$g = 5$
$g^a \bmod p = 8$
$b = 3$

$(g^a \bmod p)^b \bmod p$
$= (8)^3 \bmod 23$
$= 6$
$= g^{ab} \bmod p$

Eve
(도청자)

$P = 23$
$g = 5$
$g^a \bmod p = 8$
$g^b \bmod p = 10$

Notation

$\mathbb{Z}$ (정수 집합): {..., -2, -1, 0, 1, 2, ...}

$\mathbb{N}$ (자연수 집합): {0, 1, 2, ...}

$\mathbb{Z}_+$ (양의 정수 집합): {1, 2, 3, ...}

Integers mod N

$\mathbb{Z}_n$: {0, 1, ..., N-1} (N으로 나눈 나머지들의 집합)

참고: $\mathbb{Z}_n$은 덧셈/곱셈은 가능하지만 역원(나눗셈) 계산이 안 되는 'Ring' 구조임.

$\mathbb{Z}_n$* (정예 멤버): {a ∈ $\mathbb{Z}_n$: gcd(a, N) = 1}

N과 서로소인 애들만 따로 뽑아낸 집합 (역원을 구하기 위해).

φ(N): |$\mathbb{Z}_n$*| (서로소 집합의 크기, 오일러 파이 함수).

Division and mod

INT-DIV (정수 나눗셈): 어떤 수 a를 N으로 나누었을 때, 몫 q와 나머지 r을 반환하는 함수 ($a = qN + r$, $0 \leq r < N$).

The mod operation: $a \bmod N = r \in \mathbb{Z}_n$

Congruences (합동): $a \equiv b \pmod{N}$

"성립한다는 것은, 두 수의 차이(a - b)가 N의 배수이다!"

Groups

배경

방정식 $a \cdot x = b$ 를 풀어서 미지수 x를 구하는(복호화) 과정:

1. 미지수 x를 구하려면 a를 없애야 하니, a의 **역원(a^{-1})**을 양변에 곱해야 한다.
2. 곱했을 때 원래 상태를 유지해 주는 **항등원(id)**의 개념이 필요하다.
3. 괄호를 마음대로 쳐도 결과가 망가지지 않아야 하니 **결합법칙**이 필요하다.
4. 계산 결과가 정해둔 숫자 범위 밖으로 나가면 안 되니 **닫힘**이 필요하다.

※ 따라서 무언가를 역연산 할 수 있는 필수적인 세계를 정의하기 위해 이 4가지만 남긴 것이다.

정의

Binary operation: 연산 기호의 생김새와 상관없이 집합 안에서 원소 두 개를 골라 특정 규칙에 넣었을 때 결과값이 나오는 모든 연산.

그룹 인정 조건: 어떤 그룹 G가 group으로 인정받으려면 **closure, associativity, identity, inverse**를 모두 만족해야 한다.

양의 정수 N에 대하여, **멤버 집합 $\mathbb{Z}_n$*와 모듈로 곱셈의 조합**은 이 4가지 조건을 만족한다.

Closure (닫힘): 두 원소를 연산했을 때, 결과값도 반드시 원래 집합 G 안에 있어야 한다.

$\mathbb{Z}_{12}$ (실패): $7 \cdot 5 = 35 \notin \mathbb{Z}_{12}$

$\mathbb{Z}_{12}^*$ (성공): $5 \cdot 7 \bmod 12 = 11 \in \mathbb{Z}_{12}^*$

Associativity (결합법칙): 괄호를 앞의 두 개를 먼저 치든 뒤의 두 개를 먼저 치든 최종 결과가 같아야 한다.

$((ab \bmod N)c) \bmod N = (a(bc \bmod N)) \bmod N$

Identity (항등원): 어떤 원소 a에 연산을 취했을 때 자기 자신 a를 그대로 튀어나오게 유지시켜주는 원소(id).

곱셈 모듈로 연산에서는 **1**이 항등원임.

Inverse (역원): 모든 원소 a는 자신과 연산했을 때 결과를 항등원(1)으로 만들어주는 짝 b를 가져야 한다 ($a \cdot b \bmod N = 1$).

암호학적 의미: inverse가 **복호화 키** 역할을 한다.

Exponentiation

배경: 모듈로 N이 적용된 $\mathbb{Z}_n^*$ 특수한 환경에서도 지수 법칙들이 평소처럼 작동하는가? **그렇다!**

Multiplicative notation: a^n으로 표기.

a^0 = id (항등원 1)

a^{-1} 은 a의 역원을 뜻함

지수 법칙 ($a^{i+j} = a^i \cdot a^j$)을 **"평소처럼(as usual)"** 쓸 수 있다.

Computing Shortcuts in $\mathbb{Z}_n$

Faster way: 두 개를 곱할 때마다 바로바로 N으로 나누어 나머지를 구한다.

abc mod N = ((ab mod N)c) mod N

암호학적 의미: 계산 단계마다 모듈로로 숫자를 쳐내서, 크기를 N보다 작게 유지해야 컴퓨터가 메모리 초과 없이 연산할 수 있다.

$$5^8 \bmod 14 = (5^2)^4 \equiv 11^4 \equiv (-3)^4 \equiv 81 \equiv 11 \pmod{14}$$

Group Orders & Euler's Theorem

위수 (Order): 그룹 G 안에 들어있는 **'원소의 총개수'** |G|.

$|\mathbb{Z}_n^*| = \varphi(N)$

Euler's Theorem (오일러 정리): 집합 안의 어떤 원소 a를 골라 위수 φ(N)만큼 거듭제곱하면 결과는 무조건 항등원 1이 된다.

$$a^{(\varphi(N))} \equiv 1 \pmod{N}$$

Simplifying exponentiation

Corollary: 위수가 m일 때, $a^x = a^{(x \bmod m)}$ 이 성립한다.

정리:

밑수(Base) a: 모듈로 N으로 크기를 깎는다.

지수(Exponent) x: 모듈로 φ(N)으로 크기를 깎는다.

$$5^{74} \bmod 21 \rightarrow 74 \bmod 12 = 2 \rightarrow 5^2 \bmod 21 = 4$$

시간 측정

일반적인 알고리즘은 연산 시간을 O(1)로 보지만, 암호학의 숫자는 수백~수천 비트로 거대하다.

따라서 연산 시간을 숫자의 크기가 아니라 **비트 길이(|a|)**를 기준으로 측정한다.

EXT-GCD

Extended GCD: 역원 찾기

배경

앞서 $a \cdot b \equiv 1 \pmod{N}$ 이 되는 역원(b)을 무조건 찾을 수 있다고 배웠다.
그 역원을 찾아주는 것이 EXT-GCD이다.

Extended GCD: 역원 찾기

기존 유클리드 호제법: 두 수 a와 N의 최대공약수 $\gcd(a, N)$ 만 구해준다.

EXT-GCD(확장 유클리드 호제법): 최대공약수 r을 구해줄 뿐만 아니라,
$r = a \cdot u + N \cdot v$ 를 만족하는 계수 u와 v까지 함께 찾아준다.

Extended GCD: 역원 찾기

정의(EXT-GCD)

입력: 두 정수 a와 N

출력: (r, u, v) 세 개의 값

$$r = gcd(a, N) = a \cdot u + N \cdot v$$

a와 N의 최대공약수 r을 a의 u배와 N의 v배를 더한 형태로 표현할 수 있게 해주는 계수 u, v를 변환하는 함수이다.

Lemma(보조정리)

그렇다면 EXT-GCD는 계수 u와 v를 어떻게 찾아낼까?

$(q, r) = INT\text{-}DIV(a, N)$ 일 때, $\gcd(a, N) = \gcd(N, r)$ 이 성립한다.

거대한 숫자의 최대공약수 문제를 훨씬 작은 숫자의 문제로 축소시켜 준다. 컴퓨터는 나머지가 0이 될 때까지 이 Lemma를 반복적으로 사용하여 답을 찾아낸다.

Lemma(보조정리)

Ex) $a = 5, N = 12$ 일 때, 5의 역원 구하기
$5 \cdot u + 12 \cdot v = 1$ 을 만족하는 계수 u 찾기

1. 12를 5로 나눈다 (몫 q=2)
새 나머지 r = 2, 새 계수 u = 0 - (2·1) = -2, v = 1 - (2·0) = 1

2. 장부를 과거로 밀어낸다 (현재 장부: r=2, u=-2, v=1)

3. 5를 2로 나눈다 (몫 q=2)
새 나머지 r = 1 **(최대공약수 도달!)**
새 계수 u = 1 - (2·(-2)) = **5 (역원 발견!)**
새로운 v 계수: $v_2 = 0 - (2 \times 1) = \mathbf{-2}$

검산: 5 · (5) + 12 · (-2) = 1.

Modular Inverse (모듈로 역원)

전제 조건: $\gcd(a, N) = 1$.

수식 전개:

EXT-GCD로 $au + Nv = 1$ 도출.

모듈로 N을 씌우면 Nv는 0이 됨.

$au \equiv 1 \pmod{N}$ 이 남음.

결론: 찾아낸 계수 u 에 모듈로 N을 취한 값이 **완벽한 역원(a^{-1})**이다!

Square-and-Multiply

Square-and-Multiply

배경: 지수 n이 매우 클 때 무식하게 n번 곱하면 계산 불가능.

해결책: 지수를 이진수(Binary)로 변환하여 연산 횟수를 획기적으로 줄임 $O(|n|)$.

작동 원리 (Left-to-Right):

지수를 이진수 $b_{k-1}...b_0$ 로 변환.

Square (제곱): 비트 이동 시마다 결과값을 무조건 제곱($y \leftarrow y^2$).

Multiply (곱셈): 현재 읽은 비트가 '1'이라면 밑수 a를 한 번 더 곱함.

2^{107} mod 131 상세 연산

지수 **107**을 이진수로: **1 1 0 1 0 1 1**
비트 1 (시작): y = a = **2**
비트 1:
    Square: $2^2 = 4$
    Multiply: $4 \cdot 2 =$ **8**
비트 0:
    Square: $8^2 =$ **64**
비트 1:
    Square: $64^2 = 4096 \equiv 35 \pmod{131}$
    Multiply: $35 \cdot 2 =$ **70**

Square-and-Multiply

현재 값: **70** (다음 비트: 0 1 1)
비트 0:
    Square: $70^2 = 4900 \equiv 53 \pmod{131}$
비트 1:
    Square: $53^2 = 2809 \equiv 58 \pmod{131}$
    Multiply: $58 \cdot 2 = \mathbf{116}$
비트 1 (마지막):
    Square: $116^2 \equiv (-15)^2 = 225 \equiv 94 \pmod{131}$
    Multiply: $94 \cdot 2 = 188 \equiv \mathbf{57}$
결과: $2^{107} \equiv \mathbf{57} \pmod{131}$. 무식하게 107번 곱하는 대신 단 **제곱 6번, 곱셈 5번**으로 끝남!